VIDYAVARDHINI'S COLLEGE OF ENGINEERING & TECHNOLOGY

Department of Computer Engineering · Academic Year 2026–27

Software Engineering

Practical / Oral Examination

Solved Question Bank — Q1 to Q20

Q	Topic	Q	Topic
1	Use Case Diagram – Library Mgmt	11	Basis Path Testing – max of two numbers
2	Class Diagram – Hospital Mgmt	12	Component Diagram – Library Mgmt
3	Statechart – Online Shopping Cart	13	Process Model – Hospital Mgmt (Waterfall)
4	Activity Diagram – Student Mgmt	14	Process Model – Mobile Banking (Spiral)
5	Basic COCOMO – 20 KLOC Organic	15	Process Model – Smart Traffic (Agile)
6	Gantt Chart – Student Mgmt System	16	DFD L0 & L1 – Course Registration
7	DFD L0 & L1 – Hostel Mgmt	17	DFD L0 & L1 – Railway Reservation
8	Process Model – Online Exam (Incremental)	18	FP + COCOMO – Online Pharmacy
9	Basis Path – Even/Odd/Invalid	19	COCOMO + LOC – Smart Parking
10	White-box – Prime Number	20	Class Diagram – Travel Agency

Q1

Draw the Use Case Diagram for a Library Management System.

Step 1 – Identify actors

- **Member** (student / staff who borrows books)
- **Librarian** (manages books, members, issue / return)
- **Admin** (manages librarian accounts, reports)

Step 2 – Identify use cases

Register / Login, Search Book, Reserve Book, Issue Book, Return Book, Renew Book, Pay Fine, View Account, Manage Books, Manage Members, Generate Reports, Manage Librarian Accounts, Staff Login.

Step 3 – Relationships

- **<<include>>**: Reserve Book includes Search Book; Issue Book includes Verify Membership; Pay Fine includes Calculate Fine.
- **<<extend>>**: Calculate Fine extends Return Book only when the book is overdue.

Use Case Diagram

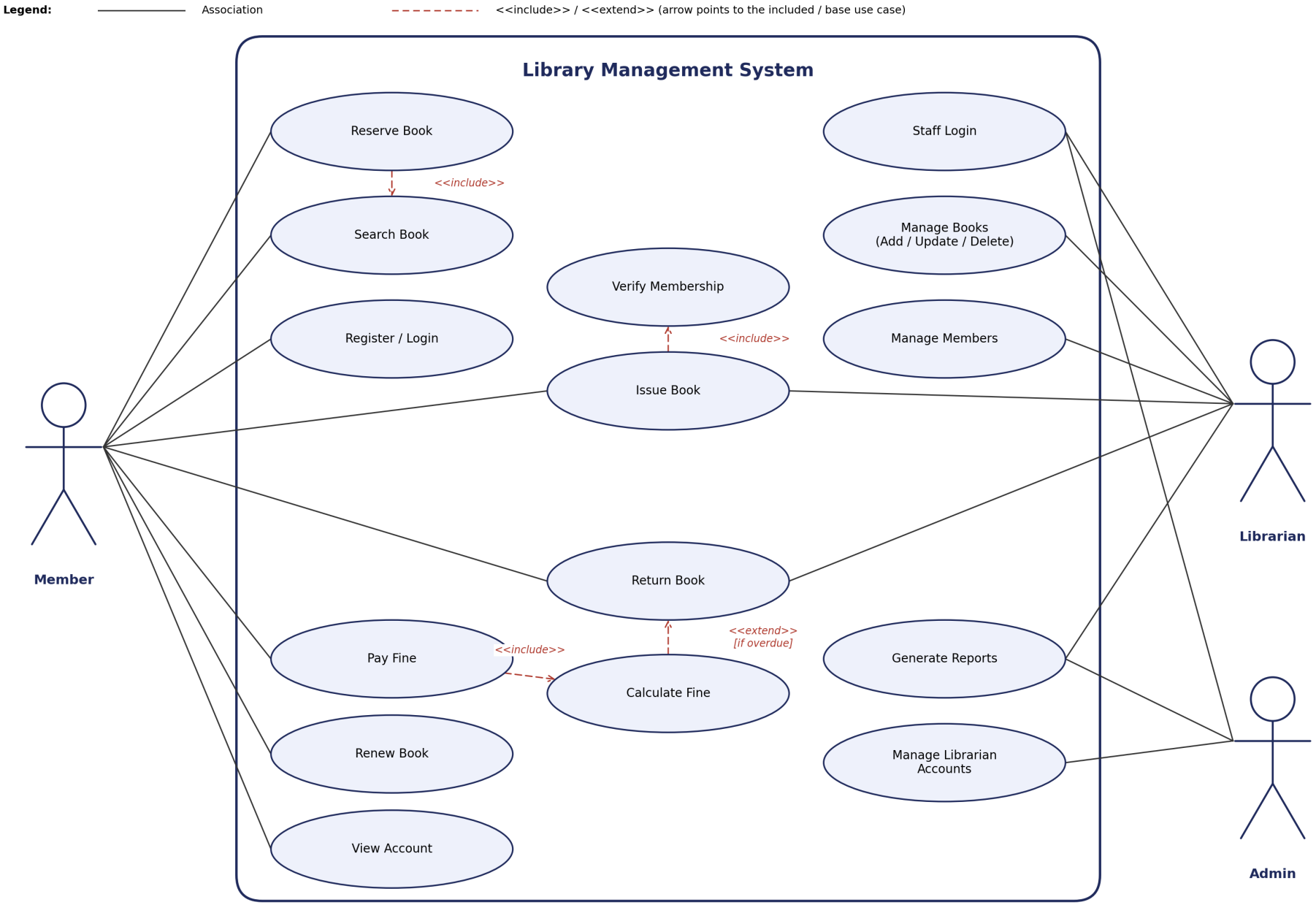


Fig. Q1 – Use Case Diagram: Library Management System

Q2

Draw the Class Diagram for a Hospital Management System.

Classes and responsibilities

Class	Key attributes	Key operations
Person (abstract)	personId, name, age, gender, phone, address	getDetails(), updateContact()
Patient	bloodGroup, admitDate	register(), bookAppointment(), viewReports()
Doctor	specialization, qualification, salary	diagnose(), prescribe(), viewSchedule()
Nurse	shift, salary	assistDoctor(), updateVitals()
Hospital / Department / Ward	name, deptName, wardNo, capacity	admitPatient(), addDoctor(), allocateBed()
Appointment, MedicalRecord, Prescription, Bill	ids, date, status, diagnosis, amount	schedule(), cancel(), addRecord(), generateBill(), payBill()

Relationships

- **Generalization**: Patient, Doctor, Nurse inherit from abstract Person.
- **Aggregation** (hollow diamond): Hospital has Departments and Wards; Department employs Doctors.
- **Composition** (filled diamond): a MedicalRecord owns its Prescriptions.
- **Association**: Patient books Appointments with Doctors; Patient has MedicalRecords, pays Bills, is admitted in a Ward.

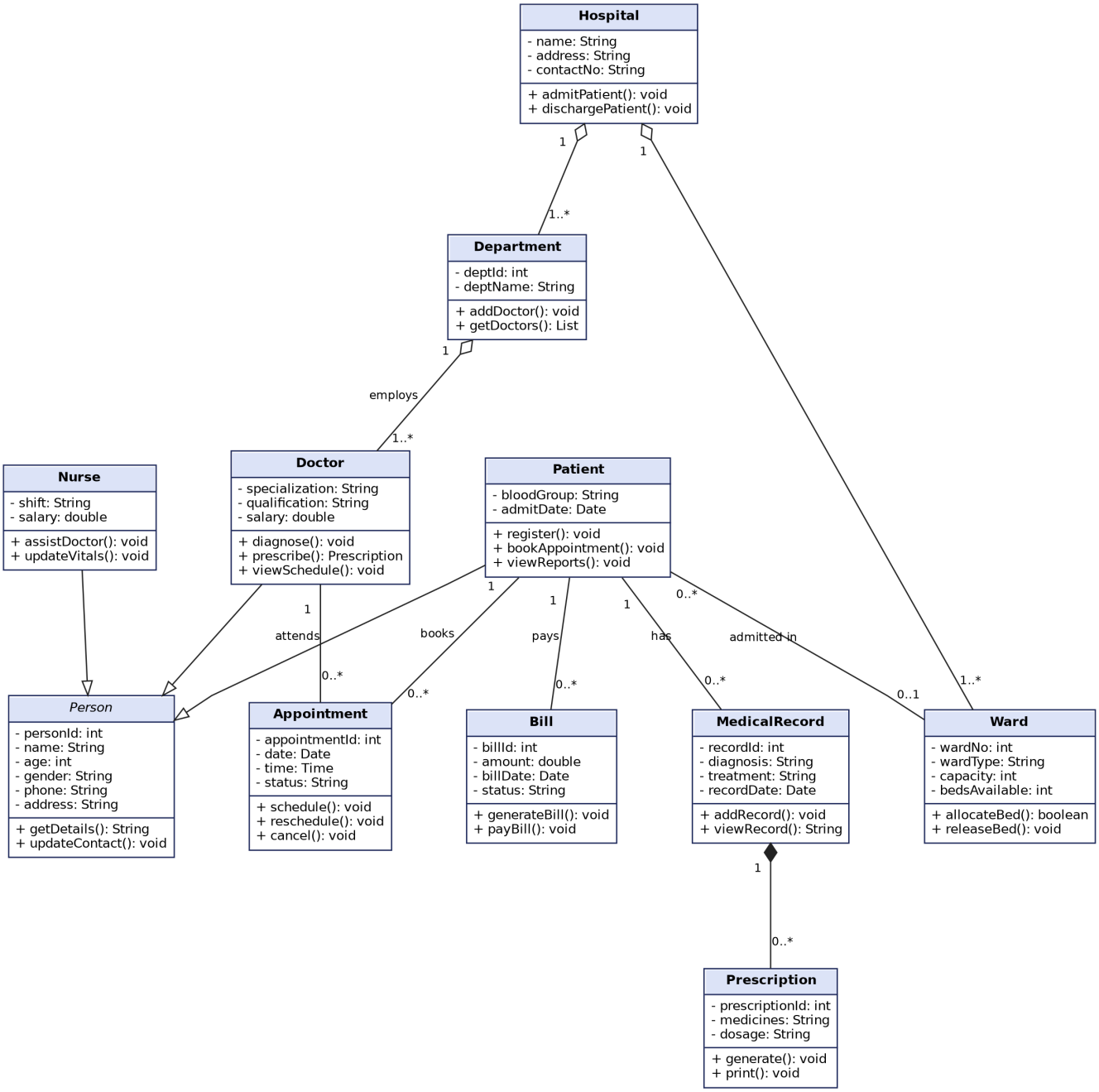


Fig. Q2 – Class Diagram: Hospital Management System

Q3

Draw the Statechart Diagram for an Online shopping cart.

States and transitions

From	Event [guard]	To
Initial	create cart	Empty
Empty	addItem()	Active
Active	addItem() / removeItem() [items > 0]	Active (self-loop)
Active	removeItem() [items = 0]	Empty
Active	proceedToCheckout()	Checkout
Checkout	editCart()	Active
Checkout	placeOrder()	Payment Pending
Payment Pending	paymentSuccess()	Order Placed
Payment Pending	paymentFailed() [retry allowed]	Checkout
Payment Pending	cancel() / timeout	Cancelled
Order Placed	dispatch()	Shipped
Order Placed	cancelOrder() [not shipped]	Cancelled
Shipped	deliver()	Delivered
Active / Empty	sessionTimeout()	Abandoned
Delivered, Cancelled, Abandoned	—	Final state

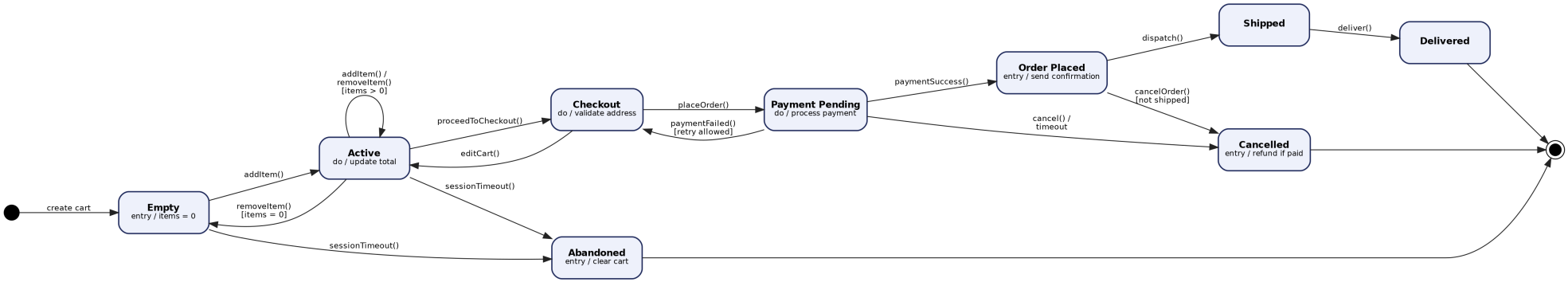


Fig. Q3 – Statechart Diagram: Online Shopping Cart

Q4

Draw the Activity Diagram for a Student Management System.

The swimlane activity diagram below follows student admission through to reports. Lanes: **Student**, **Admin / Staff**, **System**. A decision node validates the form (loop back on error) and a fork / join shows three activities that run in parallel after the course is assigned.

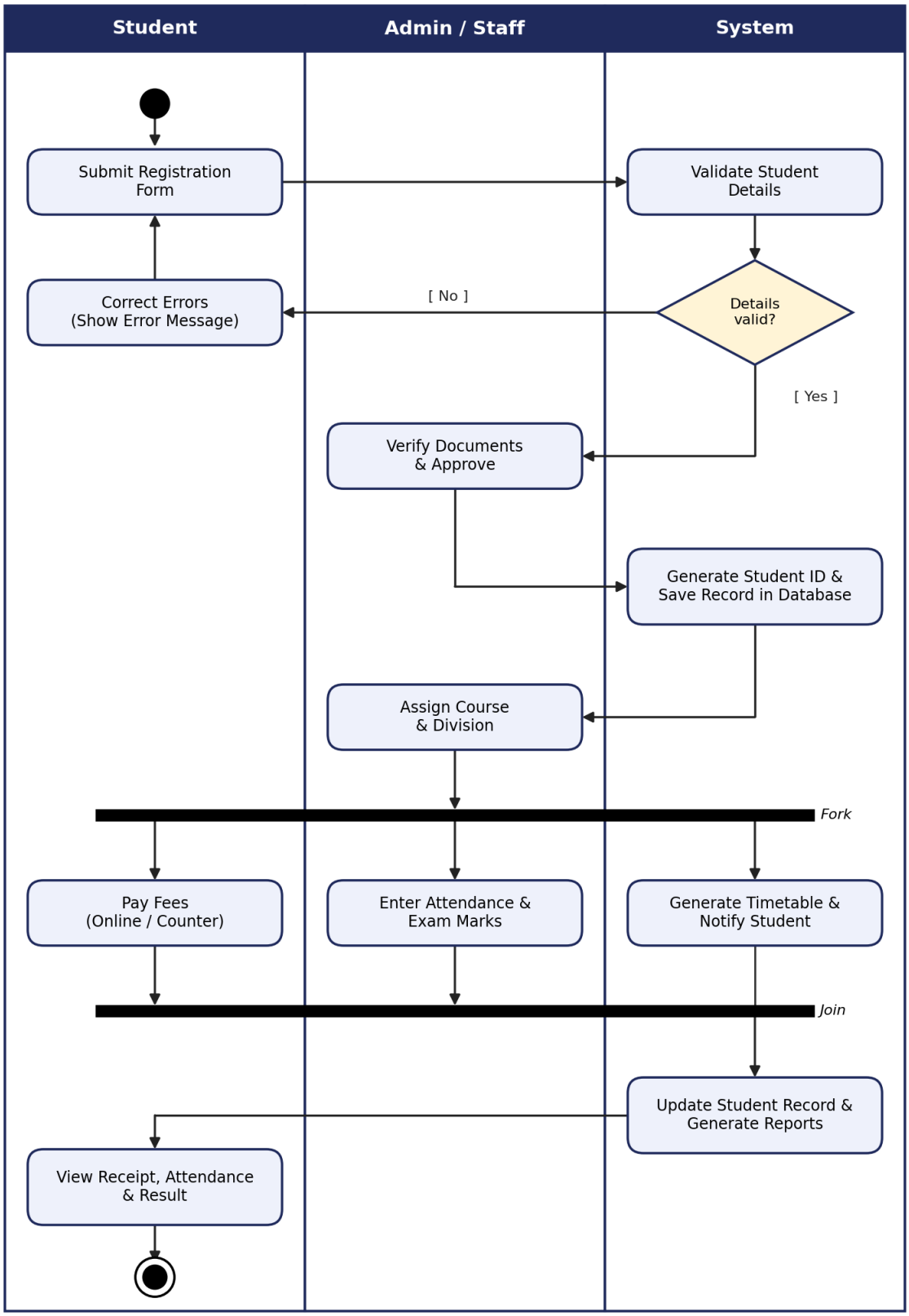


Fig. Q4 – Activity Diagram (with swimlanes): Student Management System

Q5

Estimate the development effort and time for a project of size 20 KLOC using the Basic COCOMO model. Assume the project is Organic (a=2.4, b=1.05, c=2.5, d=0.38). Calculate the estimated project cost if the development cost is ₹30,000 per person-month.

Formulae

Effort $E = a \times (KLOC)^b$ person-months
Development time $Tdev = c \times (E)^d$ months
Average staff = E / Tdev

Step 1 – Effort

$E = 2.4 \times (20)^{1.05} = 2.4 \times 23.23 =$ **55.76 person-months**

Step 2 – Development time

$Tdev = 2.5 \times (55.76)^{0.38} = 2.5 \times 4.609 =$ **11.52 months**

Step 3 – Average team size

N = E / Tdev = 55.76 / 11.52 = 4.84 ≈ **5 persons**

Step 4 – Cost

Cost = E × ₹30,000 = 55.76 × 30,000 = **₹16,72,684** (≈ ₹16.7 lakh)

Parameter	Result
Effort	55.76 person-months
Development time	11.52 months
Average staff	4.84 ≈ 5 persons
Estimated cost	₹16,72,684

Q6

A college plans to develop a Student Management System to manage student registration, attendance, examination results and fee details. (1) Identify the major project activities and assign suitable durations. (2) Define task dependencies and prepare a project schedule using a Gantt chart.

Part 1 & 2 – Activities, durations and dependencies

ID	Activity	Duration	Predecessor	Start (month)	Finish (month)
A	Requirement Analysis	1 month	—	0	1
B	Design	2 months	A	1	3
C	Coding	3 months	B	3	6
D	Testing	2 months	C	6	8
E	Deployment	1 month	D	8	9

Critical path

A → B → C → D → E = 1 + 2 + 3 + 2 + 1 = **9 months**. Every activity depends on the one before it, so all five activities lie on the critical path and have zero slack – a delay in any activity delays the whole project.

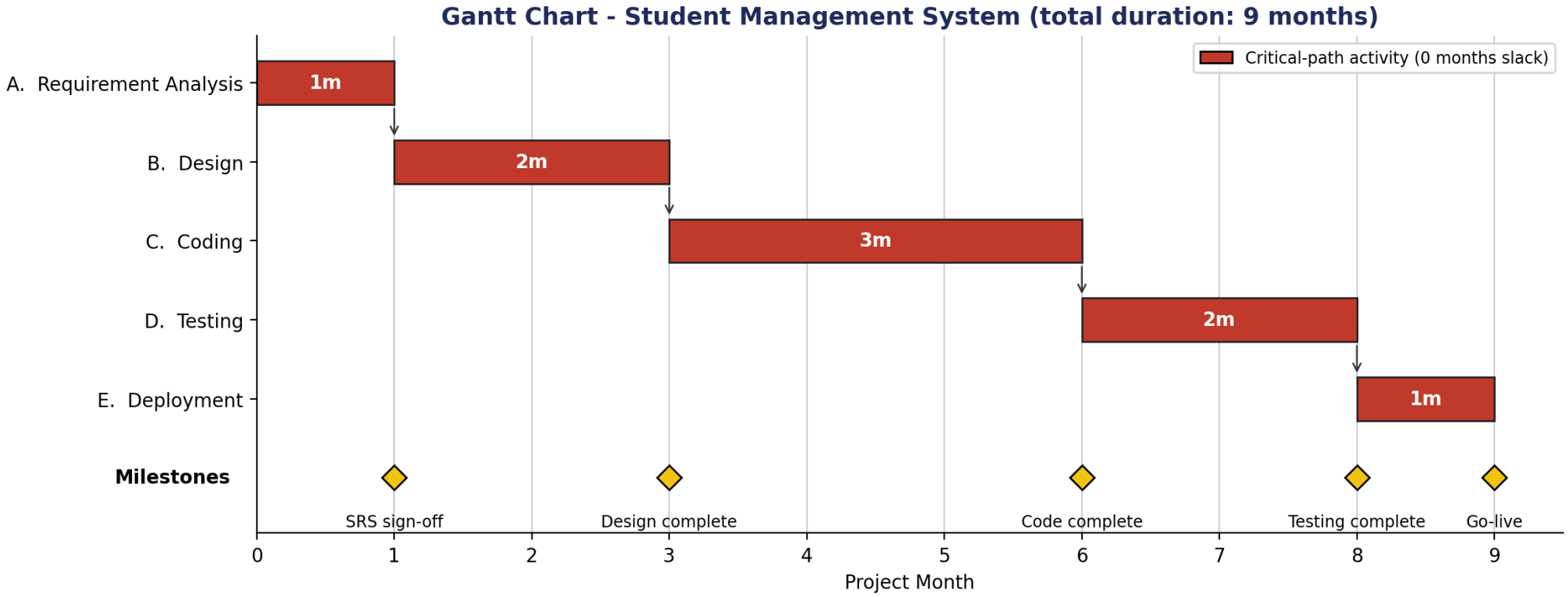


Fig. Q6 – Gantt Chart: Student Management System

Q7

Consider a Hostel Management System that manages student hostel admissions, room allocation, fee payments, and complaints. Draw the Context-Level DFD (Level 0) and the Level 1 DFD, showing major processes, data stores, external entities and data flows. Label all of them appropriately.

Notation: rectangle = external entity; circle = process; open-ended rectangle = data store; arrow = data flow.

Level 0 – Context diagram

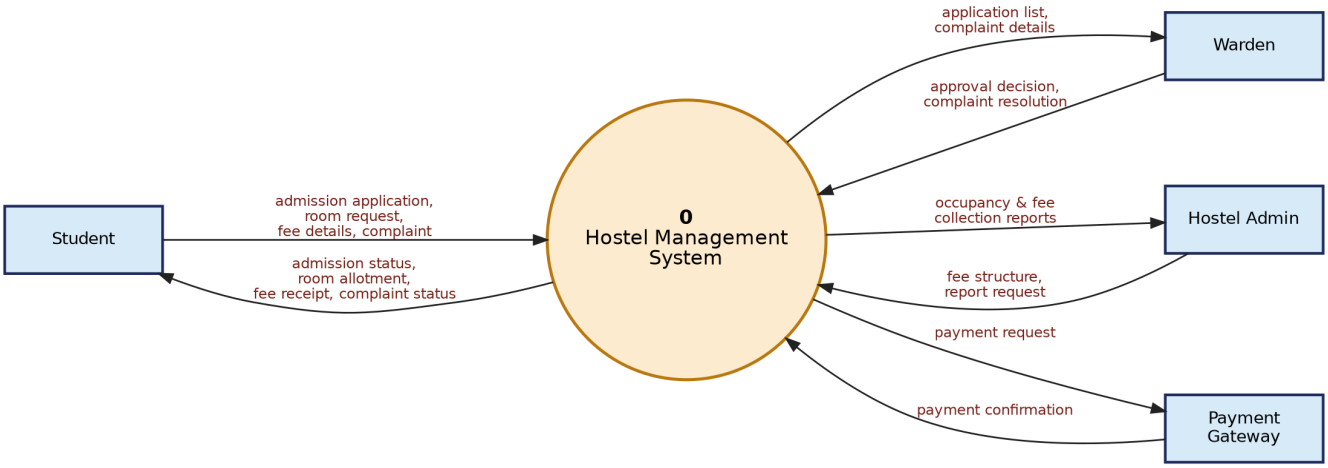


Fig. Q7(a) – Context-Level DFD (Level 0)

Level 1 – Decomposition

Item	Name
External entities	Student, Warden, Hostel Admin, Payment Gateway
Processes	1.0 Manage Admission, 2.0 Allocate Room, 3.0 Handle Complaints, 4.0 Process Fee Payment, 5.0 Generate Reports
Data stores	D1 Student Records, D2 Room Details, D3 Complaint Records, D4 Fee Records

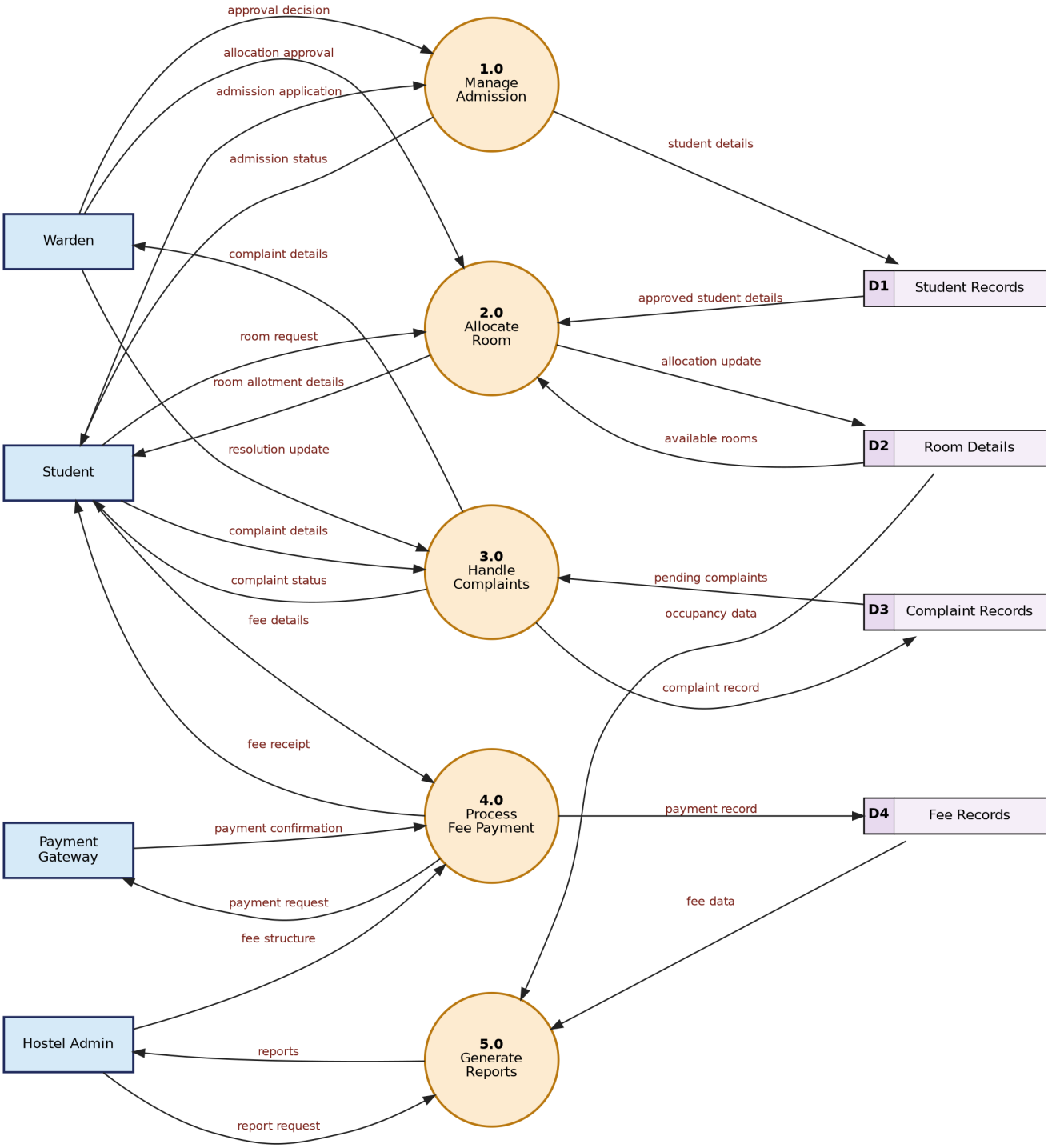


Fig. Q7(b) – Level 1 DFD: Hostel Management System

Q8

A college plans to develop an Online Examination System to conduct online tests, evaluate answers, generate results and maintain student records. The requirements may change based on feedback from faculty and students. Identify a suitable software process model and draw its diagram.

Selected model: Incremental Model

Justification

- Requirements may change with feedback from faculty and students; each increment is reviewed by users and the changes are absorbed into the next increment.
- Core functionality is delivered first and the rest is added step by step, e.g. **Increment 1:** login and online test taking; **Increment 2:** auto-evaluation and result generation; **Increment 3:** student records and reports.
- Every increment goes through analysis, design, code and test, and ends in a usable release, so users get working software early.
- Risk and cost are spread over several small deliveries; errors are found early and the first release needs fewer people and less money than a full build.

Incremental Model

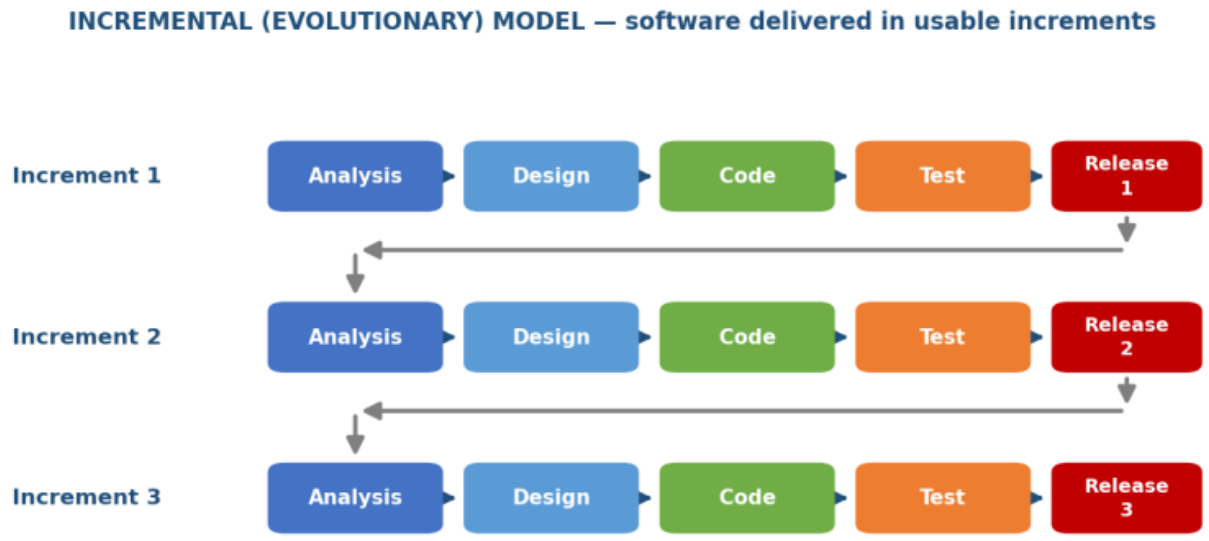


Fig. Q8 – Incremental (Evolutionary) Model

Note: the Prototyping / Agile model would also be acceptable; Incremental is chosen because the system can be delivered in usable parts while the requirements keep getting refined.

Q9

Apply Basis Path Testing to the given program and calculate the Cyclomatic Complexity.
read n; if (n > 0) { if (n % 2 == 0) print "Even" else print "Odd" } else print "Invalid"

Step 1 – Number the statements (nodes)

Node	Statement
1	read n
2	if (n > 0) *(predicate)*
3	if (n % 2 == 0) *(predicate)*
4	print "Even"
5	print "Odd"
6	print "Invalid"
7	exit / end

Step 2 – Control Flow Graph

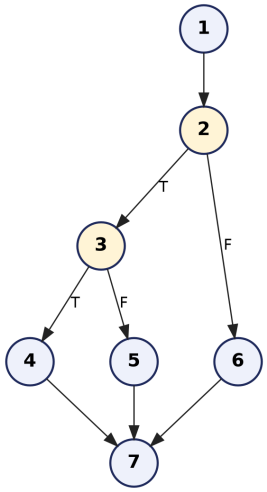


Fig. Q9 – Control Flow Graph

Edges: 1-2, 2-3, 2-6, 3-4, 3-5, 4-7, 5-7, 6-7 → **E = 8, N = 7, P = 2**

Step 3 – Cyclomatic complexity

Method	Calculation	V(G)
Regions	2 bounded regions + 1 outer	3
E − N + 2	8 − 7 + 2	3
P + 1	2 + 1	3

Step 4 – Independent paths and test cases

Path	Route	Test input	Expected output
P1	1-2-6-7	n = −5	Invalid
P2	1-2-3-4-7	n = 4	Even
P3	1-2-3-5-7	n = 7	Odd

Q10

Apply White-box testing and calculate cyclomatic complexity for the given program.
read n; i = 2; isPrime = true; while (i <= n/2) { if (n % i == 0) { isPrime = false; break } i = i + 1 } print isPrime

Step 1 – Nodes

Node	Statement
1	read n
2	i = 2; isPrime = true
3	while (i <= n/2) *(predicate)*
4	if (n % i == 0) *(predicate)*
5	isPrime = false
6	break
7	i = i + 1
8	print isPrime

Step 2 – Control Flow Graph

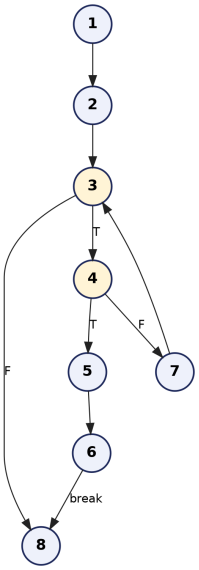


Fig. Q10 – Control Flow Graph (break jumps directly to node 8)

Edges: 1-2, 2-3, 3-4, 3-8, 4-5, 4-7, 5-6, 6-8, 7-3 → **E = 9, N = 8, P = 2**

Step 3 – Cyclomatic complexity

Method	Calculation	V(G)
Regions	2 bounded regions + 1 outer	3
E − N + 2	9 − 8 + 2	3
P + 1	2 + 1	3

Step 4 – Basis paths and test cases

Path	Route	Condition covered	Input	Expected
P1	1-2-3-8	loop never runs	n = 3	isPrime = true
P2	1-2-3-4-5-6-8	factor found, break	n = 8	isPrime = false
P3	1-2-3-4-7-3-8	no factor, loop ends normally	n = 7	isPrime = true

V(G) = 3 ≤ 10, so the module is simple and needs a minimum of **3 test cases**.

Q11

Given the following pseudo-code, apply Basis Path Testing to determine (1) Control Flow Graph, (2) Cyclomatic Complexity, (3) Independent Paths, (4) Test Cases.
read a, b; if (a > b) max = a else max = b; if (max > 0) print "Positive" else print "Non-positive"

Assumption: the second if-else is executed after the first if-else (sequential decisions).

Nodes

Node	Statement
1	read a, b
2	if (a > b) *(predicate)*
3	max = a
4	max = b
5	if (max > 0) *(predicate)*
6	print "Positive"
7	print "Non-positive"
8	exit / end

1. Control Flow Graph

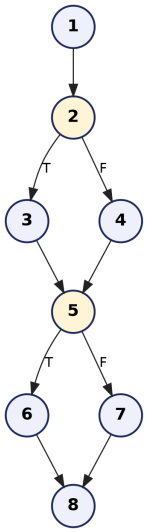


Fig. Q11 – Control Flow Graph

Edges: 1-2, 2-3, 2-4, 3-5, 4-5, 5-6, 5-7, 6-8, 7-8 → **E = 9, N = 8, P = 2**

2. Cyclomatic complexity

Method	Calculation	V(G)
Regions	2 bounded regions + 1 outer	3
E − N + 2	9 − 8 + 2	3
P + 1	2 + 1	3

3. Independent paths

Path	Route
P1	1-2-3-5-6-8
P2	1-2-4-5-6-8
P3	1-2-3-5-7-8

4. Test cases

TC	Path	Input (a, b)	Expected output
TC1	P1	a = 5, b = 3	max = 5 → Positive
TC2	P2	a = 2, b = 6	max = 6 → Positive
TC3	P3	a = −2, b = −5	max = −2 → Non-positive

The three paths cover all 9 edges. (The fourth route 1-2-4-5-7-8, e.g. a = −5, b = −2, is a linear combination of the basis paths and can be added as an extra test.)

Q12

Draw the Component Diagram for a Library Management System.

Each component exposes a **provided interface** (lollipop, solid line) and uses other components through **required interfaces** (dashed arrow to the lollipop).

Component	Provided interface	Responsibility
Authentication	IAuth	login, roles, sessions
Catalog Management	ICatalog	add / search / update books
Membership Management	IMember	member registration, validity
Circulation	ICirculation	issue, return, renew, reserve
Fine & Payment	IFine	fine calculation and payment
Notification Service	INotify	due-date and reservation alerts
Report Generator	IReport	usage and fine reports
Data Access	IData	all database operations

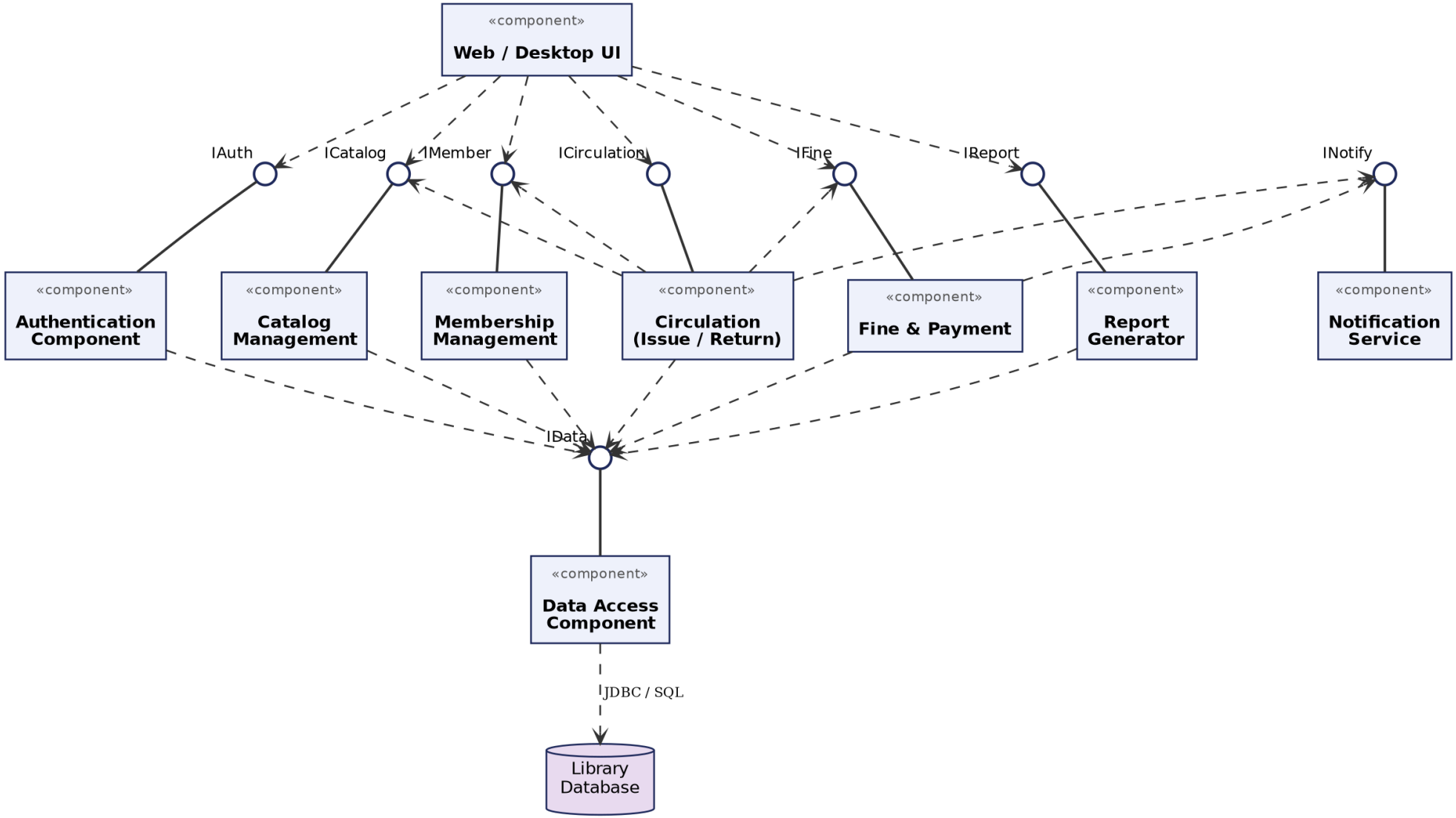


Fig. Q12 – Component Diagram: Library Management System

Q13

Hospital Management System: patient registration, appointments, billing, laboratory reports, pharmacy and electronic health records. The hospital requires a secure and reliable system with clearly defined requirements. Major requirement changes are not expected. Identify a suitable software process model and draw its diagram.

Selected model: Waterfall Model

- Requirements are clear and stable, so each phase can be finished and signed off in order.
- Strong documentation and formal reviews support the security and reliability needs of a hospital system.
- Simple to manage with well-defined milestones; testing is planned against the fixed SRS.
- Limitation: changes are costly later – acceptable here because major changes are not expected.

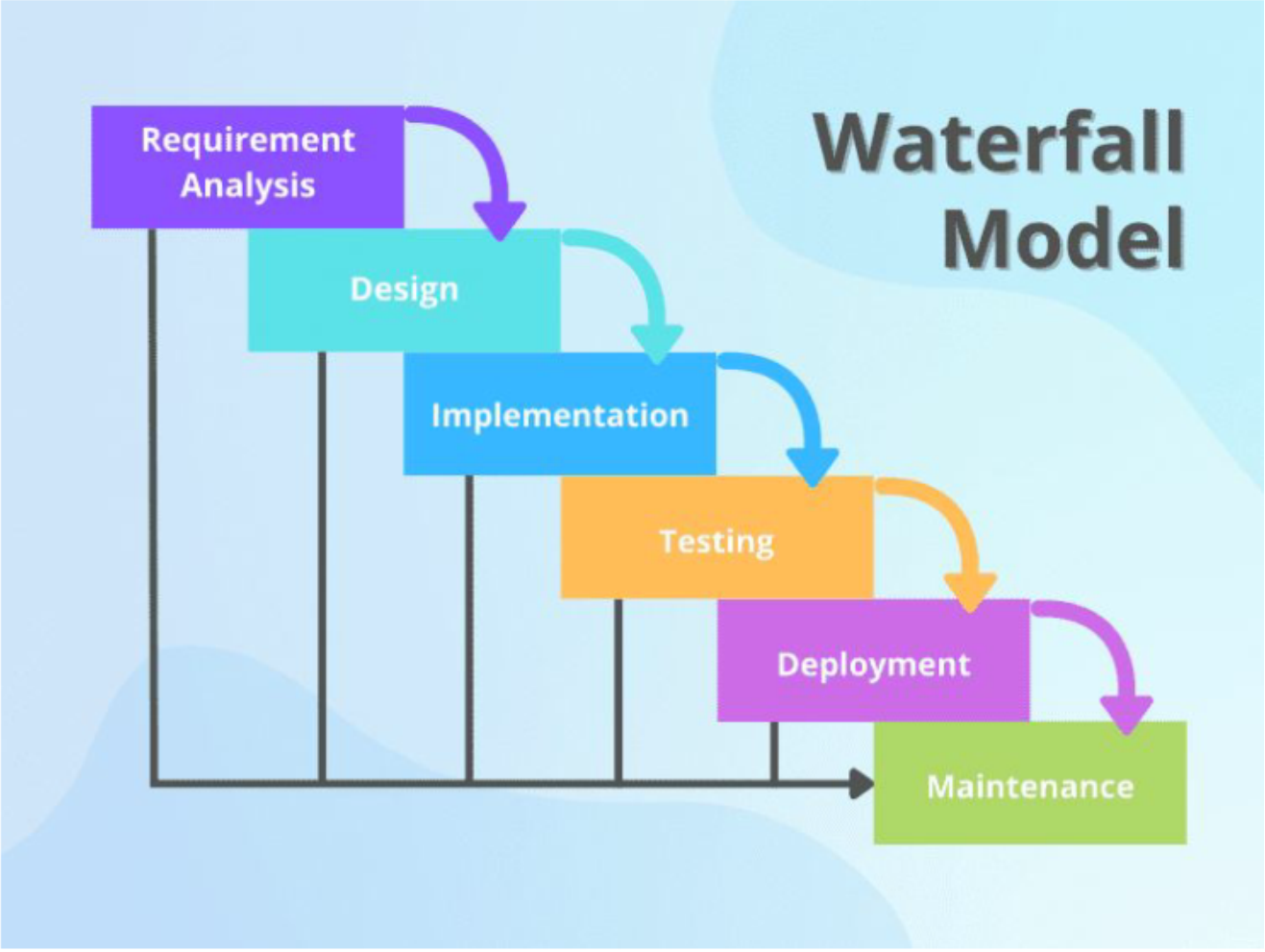


Fig. Q13 – Waterfall Model

Each phase is completed and signed off before the next begins (sequential flow).

Q14

Mobile Banking Application with fund transfer, balance enquiry, bill payment, investment services and biometric authentication. Since financial transactions are involved, the system requires high security, extensive testing and risk management. Identify a suitable software process model and draw its diagram.

Selected model: Spiral Model

- Every loop contains an explicit **risk analysis** phase – ideal for security, fraud and biometric risks.
- Prototypes are built and tested in each loop, giving extensive testing early.
- Development is incremental, so new services (e.g. investments) can be added loop by loop.
- Cost: needs risk-assessment expertise and is expensive – justified for a high-risk banking product.

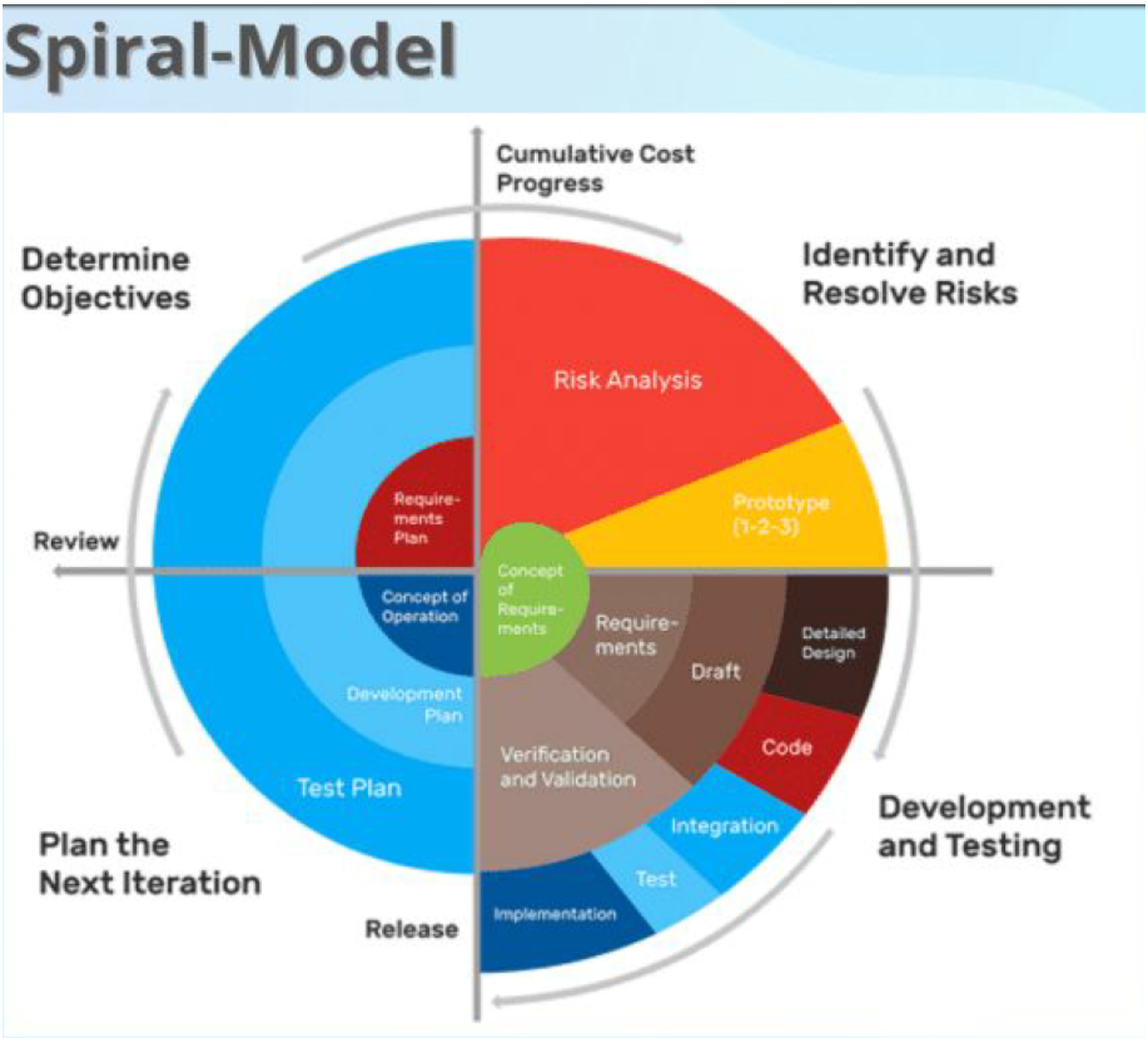


Fig. Q14 – Spiral Model

Q15

AI-Based Smart Traffic Management System to monitor traffic, optimise signal timings, detect accidents and provide real-time route suggestions. The project involves AI integration, IoT devices and evolving requirements. Identify a suitable software process model and draw its diagram.

Selected model: Agile (Scrum)

- Requirements evolve; Agile welcomes change through a re-prioritised product backlog.
- AI models and IoT integration need experimentation – short sprints give fast feedback.
- Each sprint delivers a working increment (e.g. sensor feed, signal optimiser, accident alert) that can be field-tested.
- Daily scrums and sprint reviews keep the multi-skill team (AI, IoT, backend) and the city corporation aligned.

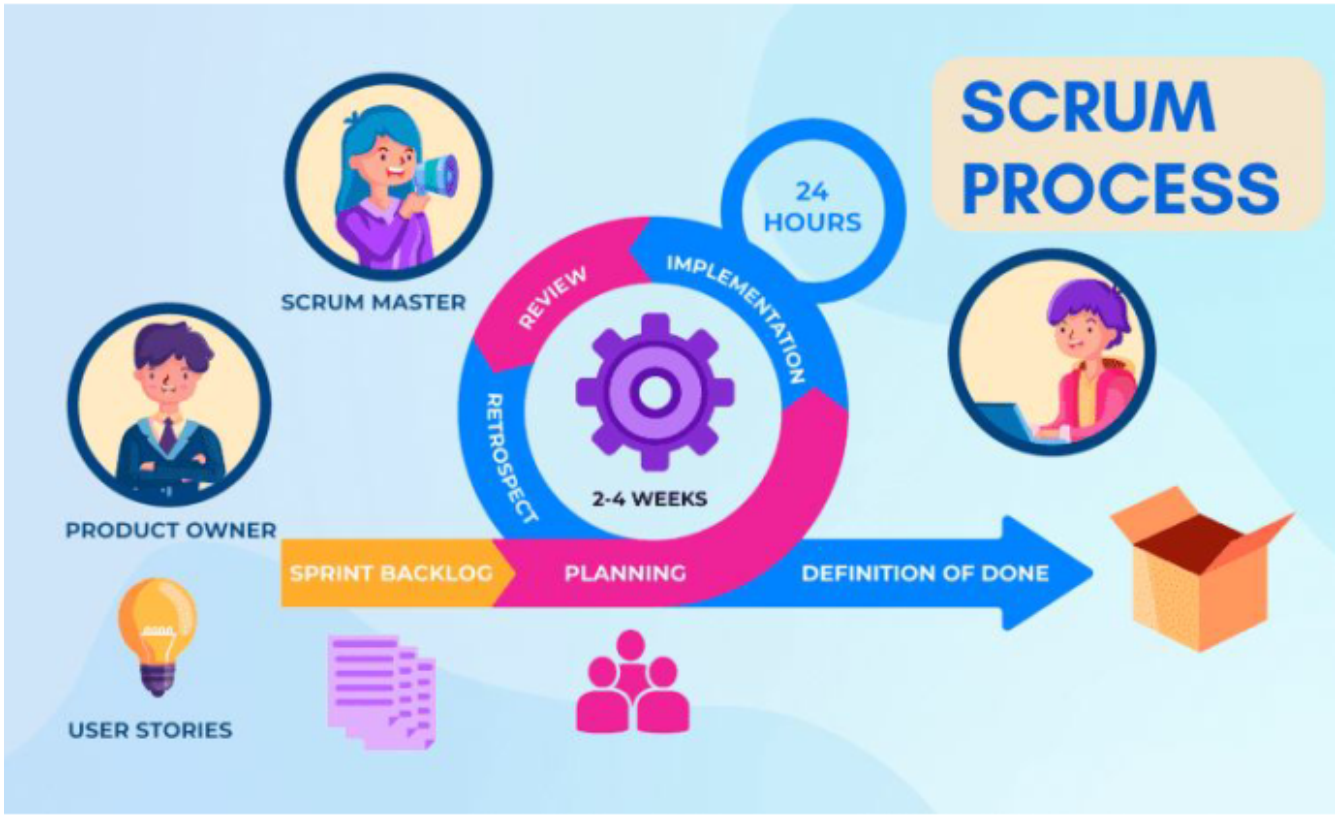


Fig. Q15 – Agile (Scrum) Model

Q16

Consider a Course Registration System. Draw the Context-Level DFD (Level 0) and the Level 1 DFD, showing major processes, data stores, external entities and data flows. Label all appropriately.

Item	Name
External entities	Student, Faculty Advisor, Registrar, Payment Gateway
Processes	1.0 Authenticate User, 2.0 Register Courses, 3.0 Process Fee Payment, 4.0 Manage Course Catalog, 5.0 Generate Reports & Timetable
Data stores	D1 Student Records, D2 Course Catalog, D3 Registration Records, D4 Fee Records

Level 0

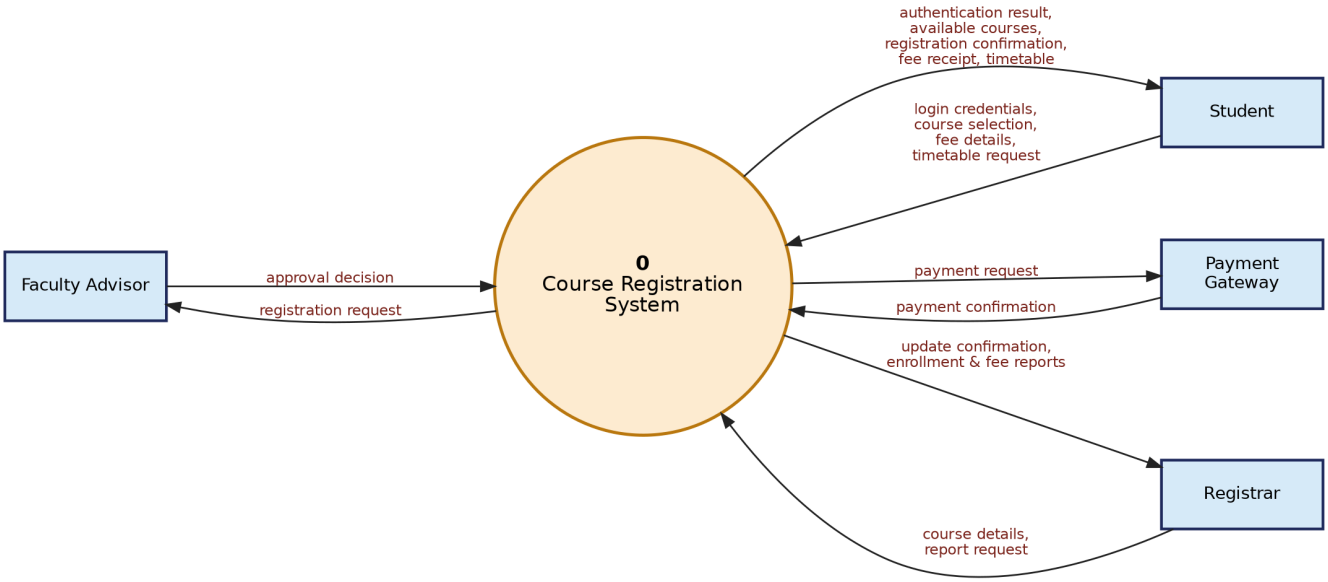


Fig. Q16(a) – Context-Level DFD

Level 1

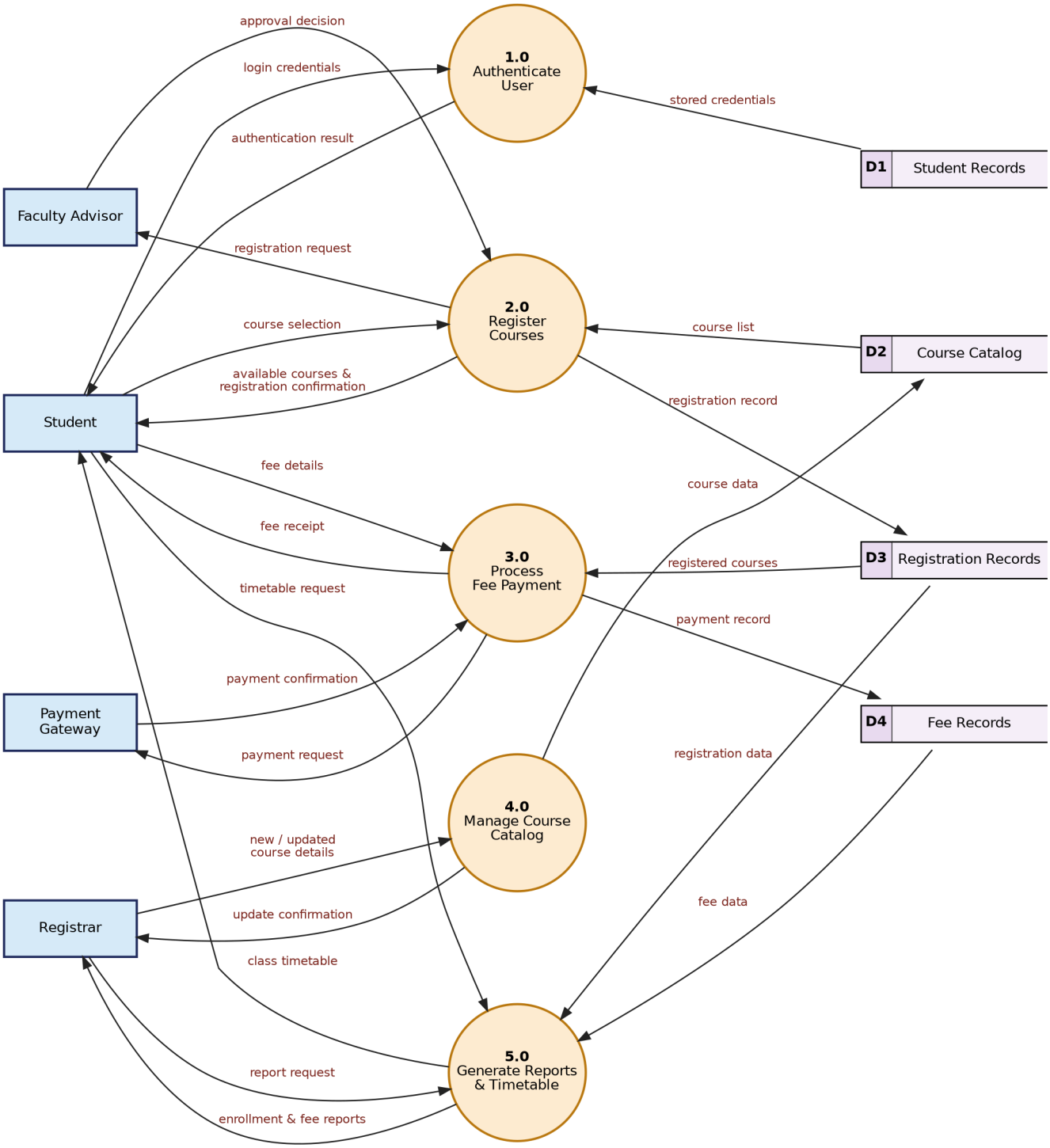


Fig. Q16(b) – Level 1 DFD: Course Registration System

Q17

Consider a Railway Reservation System. Draw the Context-Level DFD (Level 0) and the Level 1 DFD, showing major processes, data stores, external entities and data flows. Label all appropriately.

Item	Name
External entities	Passenger, Payment Gateway, Railway Admin
Processes	1.0 Search Trains & Check Availability, 2.0 Book Ticket, 3.0 Process Payment, 4.0 Cancel Ticket & Refund, 5.0 Manage Trains & Generate Reports
Data stores	D1 Train Details, D2 Booking Records, D3 Passenger Records, D4 Payment Records

Level 0

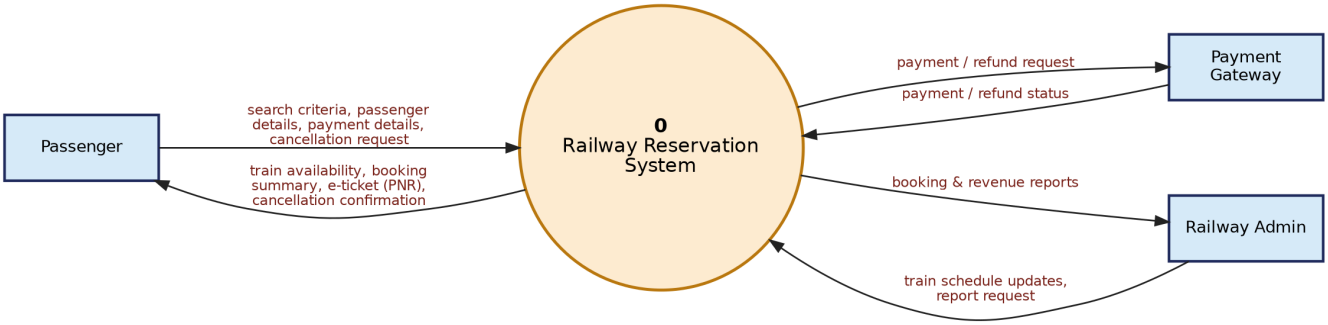


Fig. Q17(a) – Context-Level DFD

Level 1

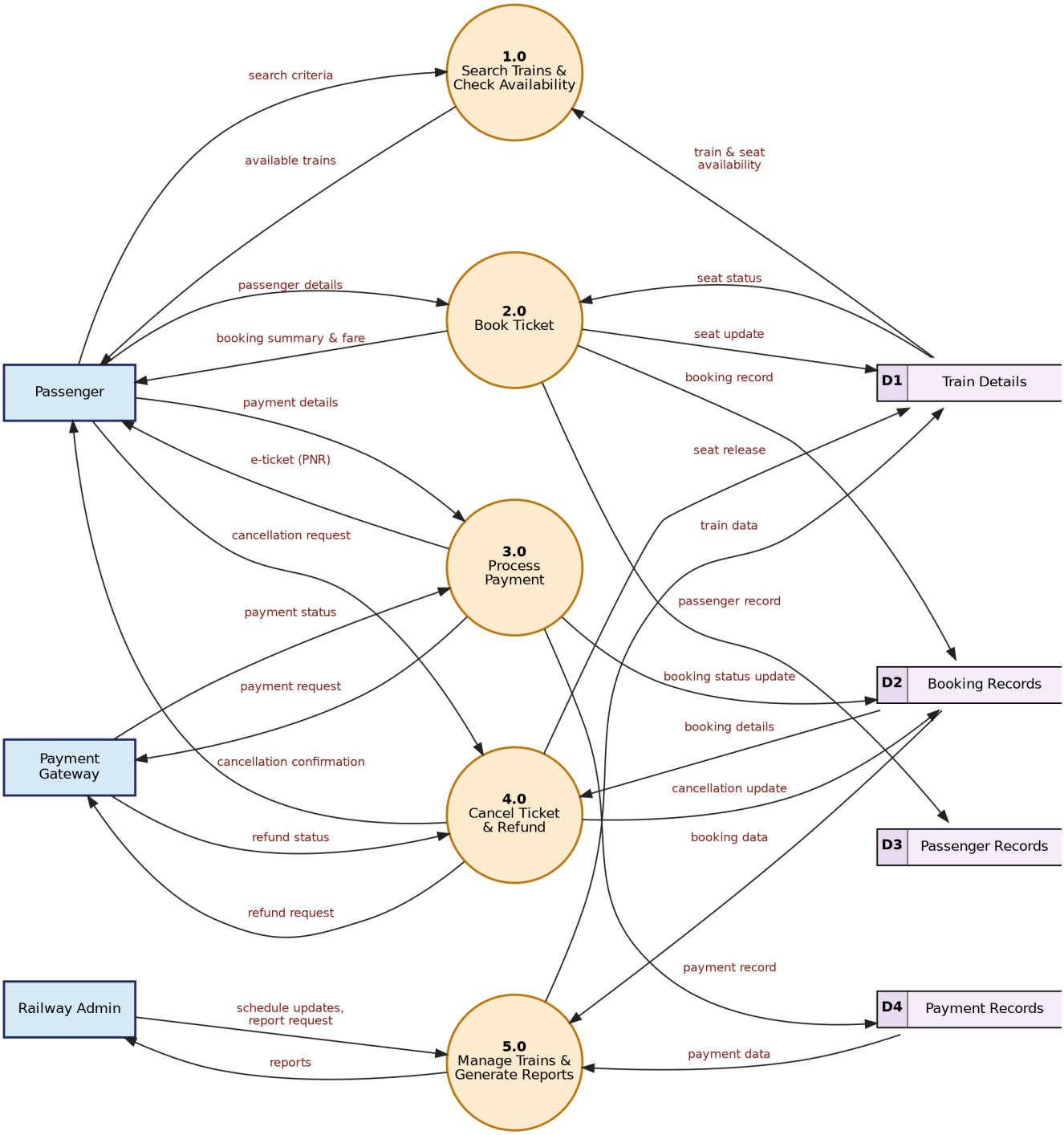


Fig. Q17(b) – Level 1 DFD: Railway Reservation System

Q18

Online Pharmacy Management System – size 32 KLOC, classified Semi-Detached. Function counts: EI = 18, EO = 14, EQ = 10, ILF = 8, EIF = 5 (all average complexity), VAF = 1.05. (1) Calculate UFP. (2) Calculate AFP. (3) Calculate effort using Basic COCOMO with a = 2.4, b = 1.05, c = 2.5, d = 0.38. (4) Discuss whether the project is suitable for the selected COCOMO category.

1. Unadjusted Function Points (average weights: EI 4, EO 5, EQ 4, ILF 10, EIF 7)

Component	Count	Weight	Product
External Inputs (EI)	18	4	72
External Outputs (EO)	14	5	70
External Inquiries (EQ)	10	4	40
Internal Logical Files (ILF)	8	10	80
External Interface Files (EIF)	5	7	35
UFP			**297**

2. Adjusted Function Points

AFP = UFP × VAF = 297 × 1.05 = **311.85 FP**
(Size check: 32,000 LOC / 311.85 FP ≈ 103 LOC per FP – realistic for a 3GL.)

3. Basic COCOMO effort (constants given in the question)

$E = 2.4 \times (32)^{1.05} = 2.4 \times 38.05 =$ **91.33 person-months**
$Tdev = 2.5 \times (91.33)^{0.38} =$ **13.90 months**
Average staff = 91.33 / 13.90 = **6.6 ≈ 7 persons**

4. Suitability of the Semi-Detached category

- Semi-Detached projects have a mix of experienced and inexperienced people, medium size and a mix of rigid and flexible requirements. The pharmacy system (prescription verification, payments, inventory, delivery tracking, regulatory rules) fits this mid-complexity profile, so the **classification is reasonable**.
- **Constants:** a = 2.4, b = 1.05, c = 2.5, d = 0.38 are the **Organic** constants. The standard Semi-Detached constants are a = 3.0, b = 1.12, c = 2.5, d = 0.35, which give $E = 3.0 \times 32^{1.12} =$ **145.51 PM**, Tdev = **14.29 months**, staff ≈ 10.
- So the effort **91.3 PM** from the given constants **under-estimates** a true Semi-Detached project by about 54 PM (59%). For planning, the Semi-Detached value (≈ 146 PM) is the safer estimate.
- Size (32 KLOC) is within the normal range for Organic / Semi-Detached models; the project is not Embedded because it has no tight hardware / real-time constraints.

Model constants	Effort (PM)	Tdev (months)	Avg staff
As given (a = 2.4, b = 1.05, c = 2.5, d = 0.38)	91.33	13.90	6.6
Standard Semi-Detached (3.0, 1.12, 2.5, 0.35)	145.51	14.29	10.2

Q19

Smart Parking Management System – estimated size 28 KLOC, Organic (a = 2.4, b = 1.05, c = 2.5, d = 0.38). (1) Calculate the project effort and development time using Basic COCOMO. (2) If the organisation's productivity is 600 LOC per person-month, calculate the effort using the LOC method.

1. Basic COCOMO

$E = 2.4 \times (28)^{1.05} = 2.4 \times 33.08 =$ **79.38 person-months**
$Tdev = 2.5 \times (79.38)^{0.38} = 2.5 \times 5.271 =$ **13.18 months**
Average staff = 79.38 / 13.18 = 6.02 ≈ 6 persons

2. LOC method

Effort = Size / Productivity = 28,000 / 600 = **46.67 person-months**

Method	Effort (PM)
Basic COCOMO	79.38
LOC method (600 LOC/PM)	46.67

Comparison: COCOMO implies a productivity of 28,000 / 79.38 ≈ 353 LOC/PM, which is lower than the assumed 600 LOC/PM. The LOC method gives a lower (optimistic) estimate; COCOMO accounts for the non-linear growth of effort with size (b > 1), so it is the more conservative figure.

Q20

Draw the Class Diagram for a Travel Agency system.

Relationship	Type	Multiplicity
Customer, TravelAgent → Person	Generalization	—
CardPayment, UPIPayment → Payment	Generalization	—
Customer makes Booking	Association	1 : 0..*
TravelAgent handles Booking	Association	1 : 0..*
Booking is for TourPackage	Association	0..* : 1
TourPackage includes Destination / Flight / Hotel	Aggregation	1 : 1..* / 0..* / 0..*
Booking is paid by Payment	Association	1 : 1..*
Booking generates Invoice	Composition	1 : 1

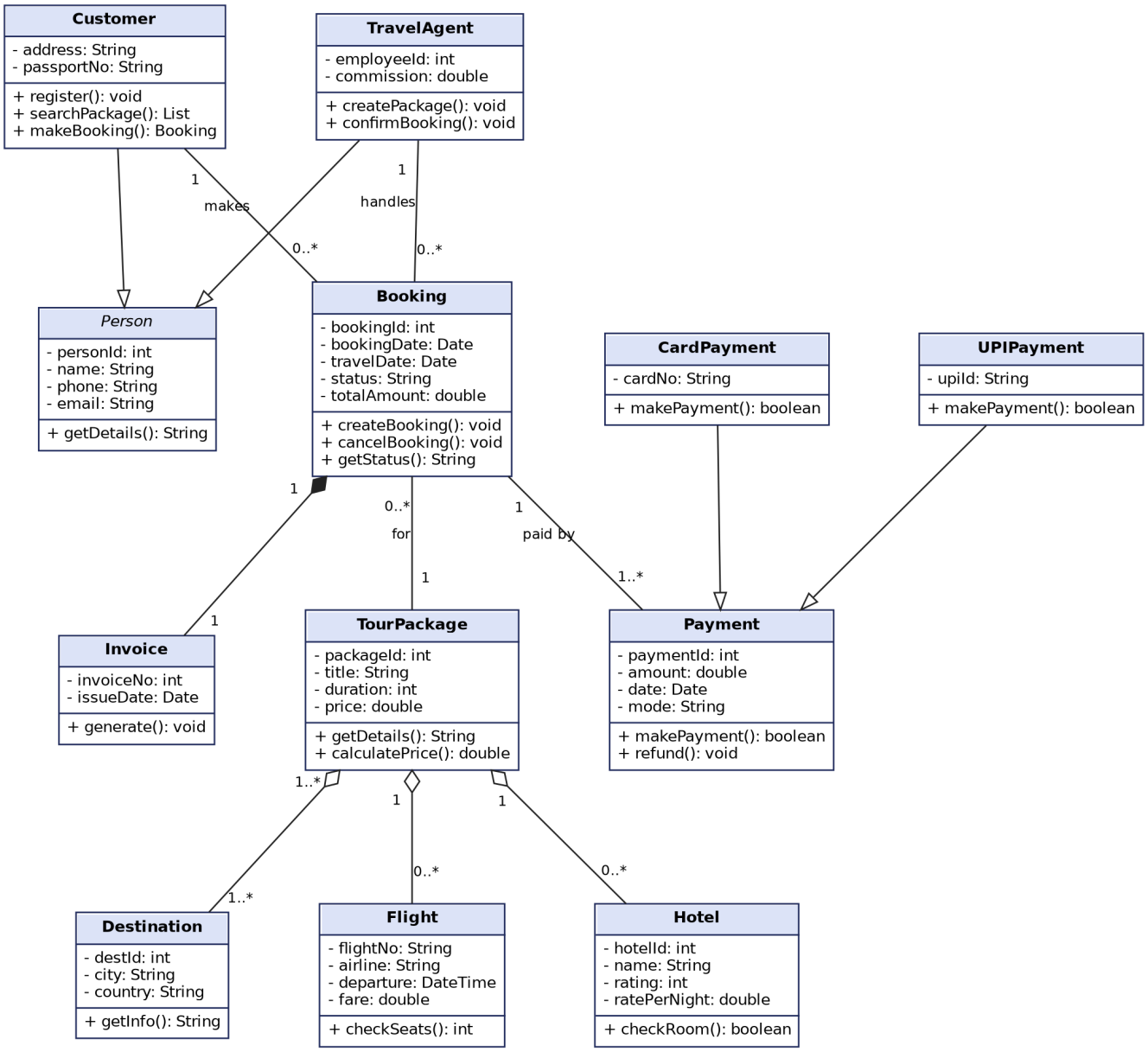


Fig. Q20 – Class Diagram: Travel Agency System